

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.6.14

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1396 营救	普及/提高-	最小瓶颈路、Dijkstra、优先队列
P4017 最大食物链计数	普及/提高-	拓扑排序、DAG 计数、动态规划
P3371 单源最短路径（弱化版）	普及/提高-	最短路、Dijkstra、手写堆
P4779 单源最短路径（标准版）	普及/提高-	最短路、Dijkstra、手写堆

（一）营救

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1396>

知识点：最小瓶颈路、Dijkstra、优先队列

题目简述

给定一张无向图，每条边有一个拥挤度。要求从 s 到 t 选一条路线，使得经过道路中的“最大拥挤度”尽可能小。

对于全部数据， $n \leq 10^4$ ， $m \leq 2 * 10^4$ ，并保证从 s 一定可以到达 t 。

1.00s, 125MB。

题目分析

1. 读题

题目要优化的不是路径总和，而是路径上的最大边权。这和普通最短路不同，更像是在所有可行路径里，寻找“最小瓶颈”。

2. 性质观察

虽然目标不是路径边权和，但它依然具有“最优子结构”：如果已经知道从 s 到 u 的最小瓶颈值，那么经过边 $u \rightarrow v$ 之后，到 v 的候选代价就是 $\max(\text{dist}[u], w)$ 。

这说明本题可以改写成一种特殊的最短路模型，只不过松弛规则不再是加法，而是“取当前路径最大边权的最小值”。

3. 程序设计思路

1. 用邻接表存储整张无向图；
2. 初始化 dist 为无穷大，令 $\text{dist}[s] = 0$ ；
3. 用小根堆维护当前瓶颈值最小的候选结点；
4. 每次取出当前 dist 最小的结点 u ；
5. 枚举它的每条边 (u, v, w) ，计算新代价 $\text{nd} = \max(\text{dist}[u], w)$ ；
6. 如果 $\text{nd} < \text{dist}[v]$ ，就更新并把 v 压入堆中；
7. 最终输出 $\text{dist}[t]$ 。

4. 数据结构与算法选择

当前代码使用的是“改造版 Dijkstra + 手写小根堆”。它和普通最短路的区别只在松弛式子：从加法改成了 $\max$ ，但整体流程仍然保持一致。

5. 时空复杂度分析

- 时间复杂度： $O((N + M) \log N)$ 。
- 空间复杂度： $O(N + M)$ 。

代码实现

混合 / 评测记录 / 评测详情

R282395763 记录详情

编程语言

C O2

代码长度

1.73KB

用时

37ms

内存

1.04MB

测试点信息

源代码

测试点信息

#1

AC

3ms/816.00KB

#2

AC

3ms/1.04MB

#3

AC

3ms/1.04MB

#4

AC

3ms/816.00KB

#5

AC

3ms/812.00KB

#6

AC

3ms/816.00KB

#7

AC

4ms/816.00KB

#8

AC

4ms/816.00KB

#9

AC

4ms/812.00KB

#10

AC

7ms/900.00KB

jsj0708Baijiahui

所属题目

P1396 营救

评测状态

Accepted

评测分数

100

提交时间

2026-06-20 12:31:16

```

#include <stdio.h>
#include <string.h>

#define MAXN 10005
#define MAXM 40005
#define INF 0x3f3f3f3f

int head[MAXN];
int to[MAXM];
int nxt[MAXM];
int w[MAXM];
int cnt;

void add(int u,int v,int val)
{
    to[++cnt]=v;
    w[cnt]=val;
    nxt[cnt]=head[u];
    head[u]=cnt;
}

int dist[MAXN];
int vis[MAXN];

typedef struct
{
    int d;
    int id;
}Node;

Node heap[200005];
int sz;

void push(int d,int id)
{
    heap[++sz].d=d;
    heap[sz].id=id;

    int i=sz;

    while(i>1)
    {
        int p=i/2;

```

```

        if(heap[p].d<=heap[i].d)
            break;

        Node t=heap[p];
        heap[p]=heap[i];
        heap[i]=t;

        i=p;
    }
}

Node pop()
{
    Node ret=heap[1];

    heap[1]=heap[sz--];

    int i=1;

    while(1)
    {
        int s=i;
        int l=i*2;
        int r=i*2+1;

        if(l<=sz && heap[l].d<heap[s].d)
            s=l;

        if(r<=sz && heap[r].d<heap[s].d)
            s=r;

        if(s==i) break;

        Node t=heap[s];
        heap[s]=heap[i];
        heap[i]=t;

        i=s;
    }

    return ret;
}

```

```

int main()
{
    int n,m,s,t;

    scanf("%d%d%d%d",&n,&m,&s,&t);

    for(int i=1;i<=m;i++)
    {
        int u,v,val;

        scanf("%d%d%d",&u,&v,&val);

        add(u,v,val);
        add(v,u,val);
    }

    memset(dist,0x3f,sizeof(dist));

    dist[s]=0;

    push(0,s);

    while(sz)
    {
        Node cur=pop();

        int u=cur.id;

        if(vis[u]) continue;

        vis[u]=1;

        for(int i=head[u];i;i=nxt[i])
        {
            int v=to[i];

            int nd=dist[u];

            if(w[i]>nd)
                nd=w[i];

            if(nd<dist[v])

```

```
        {
            dist[v]=nd;
            push(nd,v);
        }
    }

    printf("%d\n",dist[t]);

    return 0;
}
```

(二) 最大食物链计数

题目信息

题目链接：<https://www.luogu.com.cn/problem/P4017>

知识点：拓扑排序、DAG 计数、动态规划

题目简述

给定一个食物网，有向边表示“被吃”到“捕食者”的关系。要求统计从生产者到顶级消费者的最大食物链数量，并对 80112002 取模。

数据保证图中没有环。对于全部数据， $n \leq 5000$ ， $m \leq 5 * 10^5$ 。

1.00s, 125MB。

题目分析

1. 读题

所谓“最大食物链”，就是从入度为 0 的生产者出发，一直走到出度为 0 的消费者为止的一条完整链。题目要统计这样的链有多少条。

2. 性质观察

题目已经明确说明没有环，因此整张图是一张 DAG。DAG 上的路径计数通常可以用拓扑排序配合动态规划完成。

如果设 $dp[u]$ 表示“从某个生产者走到 u 的方案数”，那么：

1. 所有入度为 0 的点都可以作为链的起点，因此初始 $dp[u]=1$ ；
2. 对于每条边 $u \rightarrow v$ ，都可以把 $dp[u]$ 累加到 $dp[v]$ 上；
3. 最后把所有出度为 0 的点的 dp 值求和，就是答案。

3. 程序设计思路

1. 建立邻接表，并统计每个点的入度和出度；
2. 把所有入度为 0 的点压入队列，同时令它们的 $dp=1$ ；
3. 按拓扑顺序不断弹出队头结点；
4. 对其所有出边执行转移 $dp[v] += dp[u]$ ，并对模数取模；
5. 某个后继点入度减到 0 时，再把它压入队列；
6. 拓扑结束后，把所有出度为 0 的结点的 dp 值相加。

4. 数据结构与算法选择

本题最自然的方案就是“拓扑排序 + DAG 计数 DP”。相比 DFS 暴力枚举路径，它不会重复搜索同一段子结构，更适合 $5 * 10^5$ 条边的数据范围。

5. 时空复杂度分析

- 时间复杂度： $O(N + M)$ 。
- 空间复杂度： $O(N + M)$ 。

首页 / 评测记录 / 评测详情

R282395824 记录详情

编程语言

C O2

代码长度

1.20KB

用时

181ms

内存

4.02MB

测试点信息

源代码

测试点信息

#1

AC

3ms/788.00KB

#2

AC

3ms/788.00KB

#3

AC

3ms/788.00KB

#4

AC

3ms/788.00KB

#5

AC

8ms/812.00KB

#6

AC

9ms/1.04MB

#7

AC

24ms/1.77MB

#8

AC

16ms/1.27MB

#9

AC

45ms/2.89MB

#10

AC

67ms/4.02MB

jsj0708Baijiahui

所属题目

P4017 最大食物链计数

评测状态

Accepted

评测分数

100

提交时间

2026-06-20 12:32:03

```

#include <stdio.h>

#define N 5005
#define M 500005
#define MOD 80112002

int head[N], to[M], nxt[M], idx;
int indeg[N], outdeg[N];
int dp[N];

int q[N];
int front, rear;

void add(int u, int v)
{
    to[idx] = v;
    nxt[idx] = head[u];
    head[u] = idx++;
}

int main()
{
    int n, m;
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++)
        head[i] = -1;

    for (int i = 0; i < m; i++)
    {
        int a, b;
        scanf("%d%d", &a, &b);

        add(a, b);

        indeg[b]++;
        outdeg[a]++;
    }

    front = rear = 0;

    /* 所有生产者入队 */
    for (int i = 1; i <= n; i++)

```

```

{
    if (indeg[i] == 0)
    {
        dp[i] = 1;
        q[rear++] = i;
    }
}

/* 拓扑排序 + DP */
while (front < rear)
{
    int u = q[front++];

    for (int i = head[u]; i != -1; i = nxt[i])
    {
        int v = to[i];

        dp[v] = (dp[v] + dp[u]) % MOD;

        indeg[v]--;

        if (indeg[v] == 0)
            q[rear++] = v;
    }
}

long long ans = 0;
for (int i = 1; i <= n; i++)
{
    if (outdeg[i] == 0)
    {
        ans = (ans + dp[i]) % MOD;
    }
}

printf("%lld\n", ans);

return 0;
}

```

(三) 单源最短路径（弱化版）

题目信息

题目链接：<https://www.luogu.com.cn/problem/P3371>

知识点：最短路、Dijkstra、手写堆

题目简述

给定一个边权非负的有向图和起点 s ，要求输出 s 到所有点的最短路长度；若某点不可达，则输出 $2^{31}-1$ 。

对于全部数据， $n \leq 10^4$ ， $m \leq 5 * 10^5$ ，图中可能存在重边。

1.00s, 125MB。

题目分析

1. 读题

这是一道标准单源最短路题。虽然题面写的是“弱化版”，但边权全部非负，因此最稳妥的思路仍然是 Dijkstra。

2. 性质观察

因为边权 $w \geq 0$ ，从起点出发时，当前最短距离最小的未确定点一旦被取出，它的最短路就已经确定，这正是 Dijkstra 成立的关键前提。

题目还特别提醒了多条重边的存在，不过这不会影响算法正确性。无论两点之间有几条边，松弛时自然会保留其中更优的那一条。

3. 程序设计思路

1. 用邻接表存储整张图；
2. 初始化 $\text{dist}[s]=0$ ，其余点为无穷大；
3. 用小根堆维护当前候选的最短距离结点；
4. 每次取出堆顶 (d,u) ，若它已经过期就跳过；

- 5. 扫描 u 的所有出边，尝试用 $\text{dist}[u] + w$ 更新相邻点；
- 6. 算法结束后，依次输出所有点距离；仍为无穷大的点输出 $2^{31}-1$ 。

4. 数据结构与算法选择

当前实现是“邻接表 + 手写二叉堆 Dijkstra”。堆中结点保存 (id, dis) ，每次手动 `push` 和 `pop` 维护最小距离优先，这和使用 STL 优先队列的思路一致，只是实现更底层。

5. 时空复杂度分析

- 时间复杂度： $O((N + M) \log N)$ 。
- 空间复杂度： $O(N + M)$ 。

代码实现

评测 / 评测记录 / 评测详情

R282395901 记录详情

编程语言C++2

代码长度1.97KB

用时356ms

内存8.30MB

测试点信息源代码

测试点信息

#1 AC 4ms/2.25MB	#2 AC 4ms/2.25MB	#3 AC 5ms/2.46MB	#4 AC 5ms/2.46MB	#5 AC 19ms/4.46MB	#6 AC 19ms/4.46MB	#7 AC 19ms/4.44MB
#8 AC 94ms/8.20MB	#9 AC 93ms/8.21MB	#10 AC 94ms/8.30MB				

jsj0708Baijiahui

所属题目P3371 【模板】单源最短路径 (...)

评测状态Accepted

评测分数100

提交时间2026-06-20 12:32:51

```

#include <stdio.h>
#include <string.h>

#define N 10005
#define M 500005
#define INF 2147483647

typedef struct
{
    int to;
    int w;
    int next;
}Edge;

Edge edge[M];

int head[N];
int cnt;

long long dist[N];
int vis[N];

void add(int u,int v,int w)
{
    edge[cnt].to=v;
    edge[cnt].w=w;
    edge[cnt].next=head[u];
    head[u]=cnt++;
}

typedef struct
{
    int id;
    long long dis;
}Node;

Node heap[M];
int size;

void swap(Node *a,Node *b)
{
    Node t=*a;
    *a=*b;

```

```

        *b=t;
    }

void push(Node x)
{
    heap[++size]=x;

    int i=size;

    while(i>1)
    {
        int fa=i/2;

        if(heap[fa].dis<=heap[i].dis)
            break;

        swap(&heap[fa],&heap[i]);
        i=fa;
    }
}

```

```

Node pop()
{
    Node ret=heap[1];

    heap[1]=heap[size--];

    int i=1;

    while(1)
    {
        int smallest=i;
        int l=i*2;
        int r=i*2+1;

        if(l<=size && heap[l].dis<heap[smallest].dis)
            smallest=l;

        if(r<=size && heap[r].dis<heap[smallest].dis)
            smallest=r;

        if(smallest==i)
            break;
    }
}

```

```

        swap(&heap[i],&heap[smallest]);
        i=smallest;
    }

    return ret;
}

int main()
{
    int n,m,s;

    scanf("%d%d%d",&n,&m,&s);

    memset(head,-1,sizeof(head));

    for(int i=0;i<m;i++)
    {
        int u,v,w;

        scanf("%d%d%d",&u,&v,&w);

        add(u,v,w);
    }

    for(int i=1;i<=n;i++)
        dist[i]=INF;

    dist[s]=0;

    push((Node){s,0});

    while(size)
    {
        Node cur=pop();

        int u=cur.id;

        if(vis[u])
            continue;

        vis[u]=1;
    }
}

```



```

for(int i=head[u];i!=-1;i=edge[i].next)
{
    int v=edge[i].to;
    int w=edge[i].w;

    if(dist[v]>dist[u]+w)
    {
        dist[v]=dist[u]+w;

        push((Node){v,dist[v]});
    }
}

for(int i=1;i<=n;i++)
{
    if(dist[i]==INF)
        printf("%d ",INF);
    else
        printf("%lld ",dist[i]);
}

return 0;
}

```

（四）单源最短路径（标准版）

题目信息

题目链接：<https://www.luogu.com.cn/problem/P4779>

知识点：最短路、Dijkstra、手写堆

题目简述

给定一张带非负权的有向图，求起点 s 到所有点的最短距离，并保证从 s 可以到达任意点。

对于全部数据， $n \leq 10^5$ ， $m \leq 2 * 10^5$ ，边权可达 10^9 。

1.00s, 512MB。

题目分析

1. 读题

这道题是标准版单源最短路模板，数据规模比弱化版更大，因此算法必须在大图上仍然稳定高效。

2. 性质观察

边权仍然全部非负，所以 Dijkstra 依旧成立。不同之处在于：

1. 结点数扩大到 10^5 ；
2. 边权和路径长度都可能很大；
3. 因此不能使用邻接矩阵，也最好把距离数组开成 `long long`。

3. 程序设计思路

1. 使用链式前向星存图；
2. 初始化 $dist[s]=0$ ，其余为极大值；
3. 通过手写二叉堆不断取出当前距离最小的结点；
4. 若该点已经确定最短路，就跳过；
5. 枚举所有出边并执行普通 Dijkstra 松弛；
6. 更新成功时把新状态重新压入堆中；
7. 最后输出全部结点距离。

4. 数据结构与算法选择

当前代码使用的是“链式前向星 + 手写小根堆”的标准版 Dijkstra。由于结点和边数都更大，这种写法比朴素 Dijkstra 更适合，也能更稳定地通过模板题。

5. 时空复杂度分析

- 时间复杂度： $O((N + M) \log N)$ 。
- 空间复杂度： $O(N + M)$ 。

代码实现

首页 / 评测记录 / 评测详情

R282395943 记录详情

编程语言

C O2

代码长度

1.95KB

用时

301ms

内存

5.65MB

测试点信息

源代码

测试点信息

#1	#2	#3	#4	#5	#6	#7
AC	AC	AC	AC	AC	AC	AC
45ms/4.14MB	29ms/3.99MB	56ms/5.51MB	52ms/5.03MB	42ms/4.70MB	45ms/5.65MB	32ms/4.95MB

jsj0708Baijiahui

所属题目

P4779 【模板】单源最短路径 (...

评测状态

Accepted

评测分数

100

提交时间

2026-06-20 12:33:31

```
#include <stdio.h>
#include <string.h>

#define N 100005
#define M 200005
#define INF 0x3f3f3f3f3f3f3fLL
```

```
typedef struct
```

```
{
    int to;
    int w;
    int next;
}Edge;
```

```
Edge edge[M];
```

```
int head[N];
```

```
int cnt;
```

```
long long dist[N];
```

```
int vis[N];
```

```
void add(int u,int v,int w)
```

```
{
    edge[cnt].to=v;
    edge[cnt].w=w;
    edge[cnt].next=head[u];
    head[u]=cnt++;
}
```

```
typedef struct
```

```
{
    int id;
    long long dis;
}Node;
```

```
Node heap[M * 2];
```

```
int size;
```

```
void swap(Node *a,Node *b)
```

```
{
    Node t=*a;
    *a=*b;
```

```

    *b=t;
}

void push(Node x)
{
    heap[++size]=x;

    int i=size;

    while(i>1)
    {
        int fa=i>>1;

        if(heap[fa].dis<=heap[i].dis)
            break;

        swap(&heap[fa],&heap[i]);

        i=fa;
    }
}

Node pop()
{
    Node ret=heap[1];

    heap[1]=heap[size--];

    int i=1;

    while(1)
    {
        int l=i<<1;
        int r=l|1;
        int smallest=i;

        if(l<=size &&
            heap[l].dis<heap[smallest].dis)
            smallest=l;

        if(r<=size &&
            heap[r].dis<heap[smallest].dis)
            smallest=r;
    }
}

```

```

        if(smallest==i)
            break;

        swap(&heap[i],&heap[smallest]);

        i=smallest;
    }

    return ret;
}

```

```

int main()
{
    int n,m,s;

    scanf("%d%d%d",&n,&m,&s);

    memset(head,-1,sizeof(head));

    for(int i=0;i<m;i++)
    {
        int u,v,w;

        scanf("%d%d%d",&u,&v,&w);

        add(u,v,w);
    }

    memset(dist,0x3f,sizeof(dist));

    dist[s]=0;

    push((Node){s,0});

    while(size)
    {
        Node cur=pop();

        int u=cur.id;

        if(vis[u])
            continue;
    }
}

```

```

vis[u]=1;

for(int i=head[u];
    i!=-1;
    i=edge[i].next)
{
    int v=edge[i].to;
    int w=edge[i].w;

    if(dist[v] > dist[u] + w)
    {
        dist[v] = dist[u] + w;

        push((Node){v,dist[v]});
    }
}

for(int i=1;i<=n;i++)
    printf("%lld ",dist[i]);

return 0;
}

```